FIT5149 · Code-Reading Mock Exam

Semester 2 · Weeks 1–12 · every question gives code and asks for its output / result

Instructions

- 100 marks: **Section A** — 16 select-all multiple-choice questions (2 marks each = 32); **Section B** — 13 short-answer questions (68 marks).
- No computer is allowed: every answer can be derived by hand or with a scientific calculator. Randomness is always supplied explicitly (no unseeded random calls).
- Library behaviour is as in the unit notebooks (outputs were produced with Python 3.13, NumPy 2.5, pandas 3.0, scikit-learn 1.9, SciPy 1.18, statsmodels/patsy, PyTorch 2.15, pgmpy 1.1.2; `DiscreteBayesianNetwork`, `DiscreteMarkovNetwork`). Arrays are printed by NumPy defaults.
- For multiple-choice questions select **all** correct options; one or more may be correct.

Week	Questions
Week 1	A1
Week 1–2	A2
Week 2	A3, B1
Week 3	A4, B2
Week 4	A5, B3
Week 5	A6, A7, A8, B4
Week 6	A9, A10, B5
Week 7	A11, B6
Week 8	A12, A13, B7
Week 9	A14, B8
Week 10	A15, B9
Week 11	B10, B11, B12
Week 12	A16, B13

Section A — Multiple choice (select all correct options) · 32 marks

Question 1 · Week 1 · EDA & preprocessing **(2 marks)**

The code below is run. Which statements about what it prints are correct?

```
import numpy as np, pandas as pd

df = pd.DataFrame({'Area':  [50, 60, np.nan, 80, 70],
                   'Type':  ['A', 'B', 'A', np.nan, 'B'],
                   'Price': [200, 250, 230, 300, 280]})
df['Area'] = df['Area'].fillna(df['Area'].mean())
df['Type'] = df['Type'].fillna(df['Type'].mode()[0])
X = pd.get_dummies(df[['Area', 'Type']], drop_first=True)

print(df['Area'].tolist())
print(df['Type'].tolist())
print(X.shape, list(X.columns))
print(df.isna().sum().sum())
print(round(df['Area'].std(), 2), round(np.std(df['Area'].to_numpy()), 2))
```

A. The first printed line is [50.0, 60.0, 65.0, 80.0, 70.0].

B. The missing Type is imputed with 'B', because B appears last in the data.

C. X has shape (5, 3) because both categories A and B receive an indicator column.

D. The column list printed is ['Area', 'Type_B'].

E. The last line prints 11.18 first and 10.0 second.

Answer: .....

Question 2 · Week 1–2 · Model evaluation (cross-validation) **(2 marks)**

`neg_mse` holds the scores returned by `cross_val_score(..., scoring="neg_mean_squared_error")` for three folds. Which statements about the printed output are correct?

```
import numpy as np

neg_mse = np.array([-9.0, -16.0, -25.0])      # what cross_val_score(...,
scoring="neg_mean_squared_error") returned
rmse = np.sqrt(-neg_mse)

print(rmse)
print(rmse.mean())
print(np.sqrt((-neg_mse).mean()))
print(rmse.std())
print(rmse.std(ddof=1))
```

A. The mean of the per-fold RMSEs (4.0) is smaller than the RMSE computed from the mean MSE (4.0825).

B. Averaging the per-fold RMSEs always gives the same number as taking the square root of the averaged MSE.

C. `rmse.std()` prints 0.8165 because NumPy's default is ddof=0 (divide by n).

D. `rmse.std(ddof=1)` prints 1.0, the sample standard deviation (divide by n−1).

E. The prefix `neg_` means that the model's prediction errors are negative.

Answer: .....

Question 3 · Week 2 · Classification metrics **(2 marks)**

A logistic-regression model produced the probabilities in `prob`. Which statements are correct?

```
import numpy as np
from sklearn.metrics import confusion_matrix

y_true = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1])
prob   = np.array([0.10, 0.20, 0.30, 0.40, 0.55, 0.60, 0.35, 0.50, 0.80, 0.90])   # P(default = 1)
pred = (prob > 0.5).astype(int)
cm = confusion_matrix(y_true, pred)

print(pred)
print(cm)
print((pred != y_true).mean())
print(cm[1, 1] / cm[1].sum())
print(cm[1, 1] / cm[:, 1].sum())
pred2 = (prob >= 0.5).astype(int)
print(confusion_matrix(y_true, pred2)[1, 1] / 4)
```

A. `cm[0, 1] = 2` counts observations whose true class is 0 but which were predicted as 1.

B. The rows of the confusion matrix correspond to the predicted classes.

C. The printed error rate is 0.4.

D. The observation with probability exactly 0.50 is classified as class 1 by the first rule.

E. Replacing `>` by `>=` raises the recall of class 1 to 0.75.

Answer: .....

Question 4 · Week 3 · Non-linear methods (polynomials & splines) **(2 marks)**

Which statements about the printed output are correct?

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from patsy import dmatrix

P = PolynomialFeatures(degree=2)
print(P.fit_transform([[2, 3]]))
print(P.get_feature_names_out(['a', 'b']))
print(PolynomialFeatures(degree=2, include_bias=False, interaction_only=True).fit_transform([[2, 3]]))

x = np.linspace(0, 10, 11)
print(dmatrix("bs(x, df=6, degree=3)", {"x": x}).shape)
print(dmatrix("bs(x, knots=(3, 6), degree=3)", {"x": x}).shape)
```

A. The first printed array is [[1. 2. 3. 4. 6. 9.]].

B. With `interaction_only=True, include_bias=False` the output is [[2. 3. 4. 9.]].

C. `bs(x, df=6, degree=3)` gives 7 columns because `dmatrix` adds an intercept column to the 6 basis functions.

D. `bs(x, knots=(3, 6), degree=3)` gives 6 columns: an intercept plus degree + #knots = 5 basis functions.

E. A cubic regression spline with two knots needs 12 free coefficients, so the design matrix would have 12 columns.

Answer: .....

Question 5 · Week 4 · Support vector machines **(2 marks)**

The code was run and printed the three lines shown. Which statements are supported by the code and its output?

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

X, y = make_blobs(n_samples=200, centers=[[0, 0], [3, 3]], cluster_std=1.5, random_state=0)
for C in [0.001, 0.1, 100]:
    m = SVC(kernel='linear', C=C).fit(X, y)
    print(C, m.n_support_, round(m.score(X, y), 3))
```

Printed output:

```
0.001 [88 88] 0.92
0.1 [29 28] 0.915
100 [24 24] 0.92
```

A. For C = 0.001 the classifier uses 176 support vectors in total.

B. Increasing C from 0.001 to 100 reduces the number of support vectors, while the training accuracy barely changes.

C. With C = 100 no training point violates the margin, because the two classes are separable.

D. C penalises margin violations: a small C tolerates many violations, producing a wider margin.

E. `m.score(X, y)` reports the test accuracy.

Answer: .....

Question 6 · Week 5 · Neural networks (CNN shapes) **(2 marks)**

Which statements are correct?

```
import torch, torch.nn as nn

x = torch.randn(8, 3, 64, 64)                    # (batch, channels, height, width)
conv = nn.Conv2d(3, 16, kernel_size=5, stride=2, padding=2)
pool = nn.MaxPool2d(kernel_size=2, stride=2)
gap  = nn.AdaptiveAvgPool2d(1)

h = conv(x)
p = pool(h)
print(h.shape)
print(p.shape)
print(gap(p).shape)
print(nn.Flatten()(p).shape)
print(sum(q.numel() for q in conv.parameters()))
```

A. `h.shape` is (8, 16, 32, 32) because $\lfloor (64 + 2 \cdot 2 - 5)/2 \rfloor + 1 = 32$.

B. With stride 1 instead of 2, the output of `conv` would be (8, 16, 64, 64).

C. The last printed value is 1200 because a `Conv2d` layer has no bias term by default.

D. `nn.Flatten()(p)` has 4096 features, so a following `nn.Linear(4096, 10)` would add 40,970 parameters.

E. `gap(p)` keeps the 16 × 16 spatial grid, so its shape is (8, 16, 16, 16).

Answer: .....

Question 7 · Week 5 · Neural networks (training mechanics) **(2 marks)**

Which statements are correct?

```
import torch, torch.nn as nn

w = torch.tensor([2.0], requires_grad=True)
for _ in range(2):
    loss = (3 * w) ** 2
    loss.backward()
print(w.grad)

w.grad.zero_()
loss = (3 * w) ** 2
loss.backward()
print(w.grad)

logits = torch.tensor([[2.0, 0.0]])
target = torch.tensor([0])
ce = nn.CrossEntropyLoss()
print(round(ce(logits, target).item(), 3))
print(round(ce(torch.softmax(logits, dim=1), target).item(), 3))
```

A. The first printed line is `tensor([72.])`, because gradients accumulate across the two `backward()` calls.

B. After `w.grad.zero_()` a single backward pass gives 36.

C. `loss.backward()` updates `w` to `w − lr·grad`.

D. `CrossEntropyLoss` expects raw logits; for logits [2, 0] and target 0 it prints 0.127 = $\log(1 + e^{-2})$.

E. Feeding `torch.softmax(logits)` into `CrossEntropyLoss` is harmless; it returns the same loss.

Answer: .....

Question 8 · Week 5 · Transformers (attention) **(2 marks)**

Which statements are correct?

```
import torch, math

Q = K = V = torch.tensor([[1.0, 0.0],
                          [0.0, 1.0]])
scores = Q @ K.T / math.sqrt(Q.size(-1))
w = torch.softmax(scores, dim=-1)
print(w.round(decimals=3))
print((w @ V).round(decimals=3))

mask = torch.tensor([[1, 0],
                     [1, 1]])                    # 1 = may attend, 0 = blocked
w2 = torch.softmax(scores.masked_fill(mask == 0, float('-inf')), dim=-1)
print(w2.round(decimals=3))
print(w.sum(dim=-1))
```

A. Every row of the attention-weight matrix `w` sums to 1.

B. `w @ V` returns `V` unchanged.

C. In `w2`, row 1 is [1, 0] because the blocked position receives $-\infty$ before the softmax.

D. Dividing by $\sqrt{d_k}$ makes the scores larger, which sharpens the softmax.

E. Row 2 of `w2` is identical to row 2 of `w`, because position 2 may attend to both positions.

Answer: .....

Question 9 · Week 6 · K-means clustering **(2 marks)**

Which statements about the printed output are correct?

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[1], [2], [3], [10], [11], [12]])
km = KMeans(n_clusters=2, n_init=10, random_state=0).fit(X)

print(np.sort(km.cluster_centers_.ravel()))
print(km.inertia_)
print(km.predict([[6]]) == km.labels_[0])
print(len(set(km.labels_[:3])), len(set(km.labels_[3:])))
print(KMeans(n_clusters=3, n_init=10, random_state=0).fit(X).inertia_)
```

A. The two cluster centres are 2 and 11.

B. `inertia_` = 4.0 is the sum of squared distances of the points to their own centre.

C. The new point x = 6 is assigned to the cluster whose centre is 11.

D. With K = 3 the inertia drops to 2.5, so inertia alone can never be used to pick K.

E. Using `n_init=10` guarantees that the globally optimal partition is found.

Answer: .....

Question 10 · Week 6 · Hierarchical clustering (scipy) **(2 marks)**

Which statements about the printed output are correct?

```
import numpy as np
from scipy.spatial.distance import pdist, squareform
from scipy.cluster.hierarchy import linkage, fcluster

X = np.array([[0.], [1.], [3.], [7.], [12.]])
d = pdist(X)
Z = linkage(d, method='single')
print(len(d))
print(Z)
print(fcluster(Z, t=2, criterion='maxclust'))
Zc = linkage(X, method='complete')
print(Zc[:, 2])
print(fcluster(Zc, t=2, criterion='maxclust'))
```

A. `linkage` receives the condensed distance vector (10 = 5·4/2 entries), which is the correct input type.

B. Passing `squareform(d)` (a 5 × 5 matrix) to `linkage` would give the same `Z`.

C. In the single-linkage tree the last merge joins observation 4 (x = 12) to the other four points at height 5.

D. The final merge height of complete linkage is 12, the largest distance between a point of {0, 1, 3} and a point of {7, 12}.

E. Cutting both trees into two clusters gives the same partition.

Answer: .....

Question 11 · Week 7 · Dimension reduction (PCA & t-SNE) **(2 marks)**

Which statements about the printed output are correct?

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE

X = np.array([[2, 0], [0, 1], [-2, 0], [0, -1]], dtype=float)

pca = PCA(n_components=2).fit(X)
print(pca.explained_variance_ratio_.round(2))
print(np.abs(pca.transform([[1, 1]])).round(2))
print(pca.components_.shape)

Z = StandardScaler().fit_transform(X)
print(PCA().fit(Z).explained_variance_ratio_.round(2))
print(PCA(n_components=0.75).fit(X).n_components_)
print(hasattr(TSNE(n_components=2), 'transform'))
```

A. The first principal component explains 80% of the variance, because the variance of column 1 (8/3) is four times that of column 2 (2/3).

B. After `StandardScaler` the two components explain 50% each.

C. `PCA(n_components=0.75)` keeps one component.

D. The fitted `TSNE` object can project new observations through `transform`.

E. `pca.components_` has one row per principal component and one column per original feature.

Answer: .....

Question 12 · Week 8 · Bayesian networks (d-separation) **(2 marks)**

The network is the Student network: D → G ← I, I → S, G → L. Which statements are correct?

```
from pgmpy.models import DiscreteBayesianNetwork

m = DiscreteBayesianNetwork([('D', 'G'), ('I', 'G'), ('G', 'L'), ('I', 'S')])
print(m.is_dconnected('D', 'I'))
print(m.is_dconnected('D', 'I', observed=['L']))
print(m.is_dconnected('D', 'S'))
print(m.is_dconnected('D', 'S', observed=['G']))
print(m.is_dconnected('L', 'S', observed=['G']))
print(sorted(m.get_markov_blanket('G')))
print(sorted(m.get_markov_blanket('I')))
```

A. `D` and `I` are d-connected when nothing is observed (first printed value).

B. Observing `L` makes `D` and `I` d-connected again.

C. `D` and `S` are d-connected given `G`.

D. `L` and `S` are d-connected given `G`.

E. The Markov blanket of `G` is {D, I, L}.

Answer: .....

Question 13 · Week 8 · Bayesian networks (CPDs & factorisation) **(2 marks)**

Which statements are correct?

```
import numpy as np
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD

m = DiscreteBayesianNetwork([('D', 'G'), ('I', 'G'), ('G', 'L'), ('I', 'S')])
m.add_cpds(
    TabularCPD('D', 2, [[0.6], [0.4]], state_names={'D': ['easy', 'hard']}),
    TabularCPD('I', 2, [[0.7], [0.3]], state_names={'I': ['low', 'high']}),
    TabularCPD('G', 3, [[0.30, 0.05, 0.90, 0.50],
                        [0.40, 0.25, 0.08, 0.30],
                        [0.30, 0.70, 0.02, 0.20]],
               evidence=['I', 'D'], evidence_card=[2, 2],
               state_names={'G': ['A', 'B', 'C'], 'I': ['low', 'high'], 'D': ['easy', 'hard']}),
    TabularCPD('S', 2, [[0.95, 0.20], [0.05, 0.80]], evidence=['I'], evidence_card=[2],
               state_names={'S': ['weak', 'strong'], 'I': ['low', 'high']}),
    TabularCPD('L', 2, [[0.10, 0.40, 0.99], [0.90, 0.60, 0.01]], evidence=['G'], evidence_card=[3],
               state_names={'L': ['weak', 'strong'], 'G': ['A', 'B', 'C']}),
)
print(m.check_model())
print(round(m.get_state_probability({'D': 'hard', 'I': 'low', 'G': 'B', 'S': 'weak', 'L': 'strong'}),
4))
print(sum((c.variable_card - 1) * int(np.prod(c.cardinality[1:])) for c in m.get_cpds()))
print(2 * 2 * 3 * 2 * 2 - 1)
```

A. `check_model()` printing True means the CPDs are consistent with the graph (correct cardinalities, columns sum to 1).

B. 0.0399 equals P(D=hard)·P(I=low)·P(G=B | I=low, D=hard)·P(S=weak | I=low)·P(L=strong | G=B).

C. The network has 15 independent parameters, against 47 for an unrestricted joint table.

D. The CPD of G contributes 12 free parameters (3 states × 4 parent configurations).

E. The two numbers 15 and 47 are equal as long as the CPDs are valid probabilities.

Answer: .....

Question 14 · Week 9 · Markov networks (factors) **(2 marks)**

Which statements about the printed output are correct?

```
from pgmpy.factors.discrete import DiscreteFactor

phi1 = DiscreteFactor(['A', 'B'], [2, 2], [3, 1, 1, 3])   # values listed as (A0B0, A0B1, A1B0, A1B1)
phi2 = DiscreteFactor(['B', 'C'], [2, 2], [2, 1, 1, 2])   # values listed as (B0C0, B0C1, B1C0, B1C1)

psi = phi1 * phi2
print(sorted(psi.scope()), psi.values.shape)
tau = psi.marginalize(['B'], inplace=False)
print(sorted(tau.scope()))
print(tau.values)
print(tau.values.sum())
tau.normalize()
print(tau.values.round(3))
print(phi1.reduce([('A', 0)], inplace=False).values)
```

A. `psi = phi1 * phi2` has scope ['A', 'B', 'C'] and 8 entries.

B. `tau` was obtained by summing out A.

C. 24.0 is the partition function Z of the Markov network A − B − C with these two factors.

D. After `tau.normalize()` the entry for (A=0, C=0) equals 7/12.

E. The last line shows φ_1(A=0, B=0) and φ_1(A=0, B=1).

Answer: .....

Question 15 · Week 10 · Exact inference (variable elimination) **(2 marks)**

The factors are the scopes of the factors of a Markov network. Which statements about the printed output are correct?

```
def run(order):
    factors = [{'A', 'B'}, {'B', 'C'}, {'C', 'D'}, {'B', 'D'}, {'D', 'E'}]
    sizes = []
    for v in order:
        involved = [f for f in factors if v in f]
        scope = set().union(*involved)          # scope of psi before the sum
        sizes.append(len(scope))
        factors = [f for f in factors if v not in f] + [scope - {v}]
    return sizes

print(run('ABCDE'))
print(run('BDACE'))
print(max(run('ABCDE')), max(run('BDACE')))
```

A. With order A, B, C, D, E the largest intermediate factor has 3 variables; with B, D, A, C, E it has 4.

B. The cost of variable elimination grows exponentially with the size of the largest intermediate scope, so order A, B, C, D, E is cheaper.

C. The marginal probability computed by variable elimination depends on the elimination order.

D. Finding the elimination order with the smallest induced width is easy (polynomial time).

E. In order A, B, C, D, E eliminating B creates a new factor over {C, D}.

Answer: .....

Question 16 · Week 12 · Structure learning (I-equivalence) **(2 marks)**

Which statements about the code and its output are correct?

```
def skel(E):
    return {frozenset(e) for e in E}

def immor(E):
    pa = {}
    for u, v in E:
        pa.setdefault(v, set()).add(u)
    s = skel(E)
    return {(frozenset((a, b)), c) for c, ps in pa.items() for a in ps for b in ps
            if a < b and frozenset((a, b)) not in s}

G1 = [('X', 'Y'), ('Y', 'Z')]
G2 = [('Z', 'Y'), ('Y', 'X')]
G3 = [('X', 'Y'), ('Z', 'Y')]
G4 = [('Y', 'X'), ('Y', 'Z')]
for G in (G1, G2, G3, G4):
    print(sorted(map(sorted, skel(G))), len(immor(G)))
same = lambda A, B: skel(A) == skel(B) and immor(A) == immor(B)
print(same(G1, G2), same(G1, G3), same(G1, G4))
```

A. G1 (X → Y → Z) and G2 (Z → Y → X) are I-equivalent.

B. G3 (X → Y ← Z) is not I-equivalent to G1, because it has an immorality (v-structure) that G1 does not have.

C. G1 and G4 (X ← Y → Z) are I-equivalent.

D. Two DAGs with the same skeleton are always I-equivalent.

E. All four graphs imply $X \perp Z \mid Y$.

Answer: .....

Section B — Short answer · 68 marks

Question 17 · Week 2 · Regularisation (Lasso vs Ridge) **(6 marks)**

The 8×5 matrix X consists of five mutually orthogonal ±1 columns, so $X^TX = 8I$. The data contain no noise. State the exact output of the code, using the facts that scikit-learn minimises $(1/2n)\|y - Xw\|^2 + \alpha\|w\|_1$ for Lasso and $\|y - Xw\|^2 + \alpha\|w\|^2$ for Ridge.

```
import numpy as np
from scipy.linalg import hadamard
from sklearn.linear_model import Lasso, Ridge

H = hadamard(8).astype(float)
X = H[:, 1:6]                              # 5 columns of +-1 that are mutually orthogonal: X.T @ X =
8 * I
beta = np.array([4.0, -3.0, 2.0, 1.0, 0.5])
y = X @ beta                               # no noise

print(np.linalg.lstsq(X, y, rcond=None)[0].round(3) + 0.0)
for a in [0.5, 1.5, 3.5]:
    print('lasso', a, Lasso(alpha=a, fit_intercept=False).fit(X, y).coef_.round(3) + 0.0)
print('ridge', 8, Ridge(alpha=8, fit_intercept=False).fit(X, y).coef_.round(3) + 0.0)
```

(a) What is printed on the first line (OLS)? Why? [1]

(b) Give the three Lasso coefficient vectors printed for α = 0.5, 1.5, 3.5, showing the formula you use. [2]

(c) Give the Ridge vector for α = 8 and state how many coefficients are exactly zero in each of the four fitted models (three Lasso, one Ridge). [2]

(d) Would Ridge with a very large α set the last coefficient exactly to zero? Explain. [1]

Question 18 · Week 3 · Regression splines (degree 0) **(4 marks)**

`bs(x, knots=(5, 7), degree=0)` builds a piecewise-constant basis. Predict the output of the code. No point of `df` lies exactly at a knot.

```
import numpy as np, pandas as pd
import statsmodels.formula.api as smf

df = pd.DataFrame({'x': [1, 2, 3, 4, 6, 6.5, 8, 9, 10],
                   'y': [2, 4, 3, 3, 8, 9, 5, 7, 6]})
m = smf.ols('y ~ bs(x, knots=(5, 7), degree=0)', data=df).fit()

print(m.params.round(2).tolist())
print(m.predict(pd.DataFrame({'x': [2.5, 6.2, 9.5]})).round(2).tolist())
print(m.df_model)
print(round(m.ssr, 2))
```

(a) How many columns does the design matrix have and what do they encode? [1]

(b) What is printed on the first line (the fitted coefficients)? [1]

(c) What are the second and third lines? [1]

(d) Compute the value printed on the last line. [1]

Question 19 · Week 4 · Support vector machine **(5 marks)**

Seven observations (4 red, 3 blue) in the style of the ISLP Chapter 9 hard-margin exercise are used. `C = 1e6` makes the fit effectively a hard-margin classifier. Predict the output.

```
import numpy as np
from sklearn.svm import SVC

X = np.array([[3, 4], [2, 2], [4, 4], [1, 4], [2, 1], [4, 3], [4, 1]], dtype=float)
y = np.array(['red'] * 4 + ['blue'] * 3)
m = SVC(kernel='linear', C=1e6).fit(X, y)

print(m.classes_)
print(m.n_support_)
print(sorted(m.support_.tolist()))
print(m.coef_.round(2), m.intercept_.round(2))
print(m.predict([[3, 1], [1, 1]]))
print(m.decision_function([[3, 2]]).round(2))
```

(a) What does `classes_` print, and which class has a positive decision function? [1]

(b) Give `n_support_` and the sorted `support_` list, and identify the support vectors. [1]

(c) Give `coef_` and `intercept_`, write the separating hyperplane, and state the margin width. [2]

(d) Give the two predicted classes and the decision-function value of the point (3, 2). [1]

Question 20 · Week 5 · Neural network forward & backward pass **(7 marks)**

A network with 2 inputs, 3 ReLU hidden units and 2 softmax outputs is trained on one example with the cross-entropy loss. Predict every printed line (rounded as shown).

```
import numpy as np

x  = np.array([1.0, 2.0])
W1 = np.array([[0.5, -1.0], [1.0, 0.5], [-0.5, 1.0]])      # shape (3, 2)
b1 = np.array([0.0, -1.0, 0.5])
W2 = np.array([[1.0, 0.5, -1.0], [-1.0, 2.0, 1.0]])        # shape (2, 3)
b2 = np.array([0.0, 0.5])

u = W1 @ x + b1
h = np.maximum(0, u)
z = W2 @ h + b2
p = np.exp(z - z.max()); p /= p.sum()
y = np.array([1, 0])                                       # one-hot label
loss = -np.sum(y * np.log(p))

dz  = p - y
dW2 = np.outer(dz, h)
dh  = W2.T @ dz
du  = dh * (u > 0)
dW1 = np.outer(du, x)

print(u, h, z)
print(p.round(3), round(loss, 3), np.argmax(p))
print(dW2.round(3) + 0.0)
print(dW1.round(3) + 0.0)
print(W1.size + b1.size + W2.size + b2.size)
```

(a) First line: u, h, z. [2]

(b) Second line: probabilities, loss, predicted class; is the example classified correctly? [2]

(c) The two gradient matrices dW2 and dW1. Which row of dW1 is zero and why? [2]

(d) The last line (number of parameters). [1]

Question 21 · Week 6 · Hierarchical clustering (average linkage) **(5 marks)**

Perform the computation by hand and predict the output.

```
import numpy as np
from scipy.spatial.distance import squareform
from scipy.cluster.hierarchy import linkage, fcluster

D = np.array([[0, 2, 6, 10, 9],
              [2, 0, 5,  9, 8],
              [6, 5, 0,  4, 5],
              [10, 9, 4, 0, 3],
              [9, 8, 5,  3, 0]], dtype=float)
Z = linkage(squareform(D), method='average')
print(Z.round(2))
print(fcluster(Z, t=2, criterion='maxclust'))
print(fcluster(Z, t=3.0, criterion='distance'))
```

(a) Reproduce the four rows of Z (children, height, size). Show the updated dissimilarities you use. [3]

(b) Give the partition printed by `fcluster(Z, t=2, criterion='maxclust')`. [1]

(c) Give the partition printed with `t=3.0, criterion='distance'`. [1]

Question 22 · Week 7 · PCA from first principles **(4 marks)**

Predict the output of the code.

```
import numpy as np

X = np.array([[1, 1], [-1, -1], [2, -2], [-2, 2]], dtype=float)
Xc = X - X.mean(axis=0)
S = Xc.T @ Xc / (len(X) - 1)
vals, vecs = np.linalg.eigh(S)
order = np.argsort(vals)[::-1]
vals, vecs = vals[order], vecs[:, order]

print(vals.round(3))
print((vals / vals.sum()).round(3))
scores = Xc @ vecs[:, 0]
print(np.abs(scores).round(3))
recon = np.outer(scores, vecs[:, 0])
print(round(float(np.sum((Xc - recon) ** 2)), 3))
```

(a) The eigenvalues printed on the first line. [1]

(b) The explained-variance ratios. [1]

(c) The absolute scores on PC1. [1]

(d) The reconstruction error on the last line. How is it related to the eigenvalues? What would change if the lines that sort `vals` were deleted? [1]

Question 23 · Week 8 · Bayesian network inference (pgmpy) **(6 marks)**

The network is the "cancer" network: Pollution, Smoker → Cancer → Xray, Dyspnoea. State orders: Pollution [low, high]; Smoker, Cancer and Dyspnoea [T, F]; Xray [pos, neg]. Predict the printed output.

```
import numpy as np
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

m = DiscreteBayesianNetwork([('Pollution', 'Cancer'), ('Smoker', 'Cancer'),
                             ('Cancer', 'Xray'), ('Cancer', 'Dyspnoea')])
m.add_cpds(
    TabularCPD('Pollution', 2, [[0.9], [0.1]], state_names={'Pollution': ['low', 'high']}),
    TabularCPD('Smoker', 2, [[0.3], [0.7]], state_names={'Smoker': ['T', 'F']}),
    TabularCPD('Cancer', 2, [[0.03, 0.05, 0.001, 0.02], [0.97, 0.95, 0.999, 0.98]],
               evidence=['Smoker', 'Pollution'], evidence_card=[2, 2],
               state_names={'Cancer': ['T', 'F'], 'Smoker': ['T', 'F'], 'Pollution': ['low',
'high']}),
    TabularCPD('Xray', 2, [[0.9, 0.2], [0.1, 0.8]], evidence=['Cancer'], evidence_card=[2],
               state_names={'Xray': ['pos', 'neg'], 'Cancer': ['T', 'F']}),
    TabularCPD('Dyspnoea', 2, [[0.65, 0.3], [0.35, 0.7]], evidence=['Cancer'], evidence_card=[2],
               state_names={'Dyspnoea': ['T', 'F'], 'Cancer': ['T', 'F']}),
)
ve = VariableElimination(m)
q1 = ve.query(['Cancer'], evidence={'Smoker': 'T', 'Pollution': 'high'}, show_progress=False)
q2 = ve.query(['Cancer'], evidence={'Smoker': 'T', 'Pollution': 'high', 'Xray': 'pos', 'Dyspnoea':
'T'}, show_progress=False)
q3 = ve.query(['Cancer'], evidence={'Xray': 'pos'}, show_progress=False)
q4 = ve.query(['Xray', 'Dyspnoea'], evidence={'Cancer': 'T'}, show_progress=False)
print(q1.state_names['Cancer'], np.round(q1.values, 4))
print(np.round(q2.values, 4))
print(np.round(q3.values, 4))
print(np.round(q4.values, 4))
print(m.is_dconnected('Xray', 'Dyspnoea'), m.is_dconnected('Xray', 'Dyspnoea', observed=['Cancer']))
```

(a) Line 1 (q1): P(Cancer | Smoker=T, Pollution=high). [1]

(b) Line 2 (q2): show the Bayes calculation. [2]

(c) Line 3 (q3): P(Cancer | Xray=pos). Compare with the prior P(Cancer=T) and comment. [1]

(d) Line 4 (q4): the 2×2 table, and the independence it reveals. [1]

(e) The last line (two booleans) and its meaning. [1]

Question 24 · Week 9 · Conditional random field (brute force vs variable elimination) **(6 marks)**

A three-token linear-chain CRF with labels {O, P} defines score(y) = $\prod_t$ emit[t, y_t] · $\prod_t$ trans[y_{t-1}, y_t]. Predict the output of the code.

```
import itertools
import numpy as np

emit  = np.array([[4, 1], [2, 3], [1, 5]], dtype=float)    # emit[t, label],  label 0 = O, 1 = P
trans = np.array([[2, 1], [1, 3]], dtype=float)            # trans[prev, cur]

def score(y):
    s = np.prod([emit[t, y[t]] for t in range(3)])
    s *= np.prod([trans[y[t - 1], y[t]] for t in range(1, 3)])
    return s

scores = {y: score(y) for y in itertools.product([0, 1], repeat=3)}
Z = sum(scores.values())
best = max(scores, key=scores.get)

print(len(scores), Z)
print(best, scores[best], round(scores[best] / Z, 4))
print(round(sum(v for y, v in scores.items() if y[1] == 1) / Z, 4))
```

(a) What is the first number on line 1, and what does Z equal? [2]

(b) Give the best label sequence, its score and its probability (line 2). [1]

(c) What is printed on line 3? Say which probability it is. [1]

(d) Show that Z can be obtained by sum-product variable elimination in the order Y3, Y2, and compare the cost with brute force. [2]

Question 25 · Week 10 · Variable elimination on a chain **(5 marks)**

A chain A → B → C → D has binary variables. Predict every printed line.

```
import numpy as np

pA   = np.array([0.6, 0.4])
pB_A = np.array([[0.7, 0.3], [0.2, 0.8]])      # rows: A, columns: B
pC_B = np.array([[0.9, 0.1], [0.4, 0.6]])      # rows: B, columns: C
pD_C = np.array([[0.5, 0.5], [0.1, 0.9]])      # rows: C, columns: D

tau1 = pA @ pB_A
tau2 = tau1 @ pC_B
pD   = tau2 @ pD_C
print(tau1, tau2, pD)

m = np.einsum('a,ab,bc,c->a', pA, pB_A, pC_B, pD_C[:, 1])
print(m.round(3), m.sum().round(3))
print((m / m.sum()).round(4))
```

(a) Line 1: tau1, tau2 and pD. State what each vector represents. [2]

(b) Lines 2 and 3: the vector `m`, the sum, and the posterior P(A | D=1). [2]

(c) Count the scalar multiplications needed for the three vector–matrix products versus enumerating all joint entries; what is the general complexity for a chain of n variables with k states? [1]

Question 26 · Week 11 · Forward sampling and rejection sampling **(5 marks)**

The uniform draws are supplied in `U`, one row per sample; each variable uses the next number of its row. `draw` returns the first state k with u ≤ cumulative probability. Predict the output.

```
import numpy as np

P_G = {(0, 0): [.30, .40, .30], (0, 1): [.05, .25, .70],    # key = (I, D); entries over G = 0, 1, 2
       (1, 0): [.90, .08, .02], (1, 1): [.50, .30, .20]}
P_S1 = {0: 0.05, 1: 0.80}                                   # P(S = 1 | I)
P_L1 = {0: 0.90, 1: 0.60, 2: 0.01}                          # P(L = 1 | G)

def draw(probs, u):                 # first state k with u <= cumulative probability
    return int(np.argmax(u <= np.cumsum(probs)))

def forward_sample(u):              # order D, I, G, S, L
    d = draw([0.6, 0.4], u[0])
    i = draw([0.7, 0.3], u[1])
    g = draw(P_G[(i, d)], u[2])
    s = draw([1 - P_S1[i], P_S1[i]], u[3])
    l = draw([1 - P_L1[g], P_L1[g]], u[4])
    return d, i, g, s, l

U = [[0.31, 0.18, 0.60, 0.92, 0.45],
     [0.75, 0.55, 0.20, 0.10, 0.95],
     [0.10, 0.90, 0.50, 0.60, 0.05],
     [0.80, 0.20, 0.40, 0.77, 0.15]]
samples = [forward_sample(u) for u in U]
kept = [s for s in samples if s[3] == 1 and s[4] == 0]      # evidence S = 1, L = 0

for s in samples: print(s)
print(kept)
print(len(kept) / len(samples))
```

(a) Give the four printed samples and show how the first one is generated. [2]

(b) What are the last two lines? [1]

(c) Compute exactly P(S = 1, L = 0). What is the expected number of accepted samples out of 1000 forward samples, and why is this approach inefficient when the evidence is rare? [2]

Question 27 · Week 11 · Gibbs sampling with evidence **(5 marks)**

Gibbs sampling is run in the Student network with evidence S = 1 and L = 0 held fixed. The state is (D, I, G) and each variable is resampled in the order D, I, G. Predict the output.

```
import numpy as np

PD, PI = np.array([0.6, 0.4]), np.array([0.7, 0.3])
PG  = {(0, 0): [.30, .40, .30], (0, 1): [.05, .25, .70], (1, 0): [.90, .08, .02], (1, 1): [.50, .30,
.20]}  # key (i, d)
PS1 = np.array([0.05, 0.80])            # P(S = 1 | I)
PL0 = np.array([0.10, 0.40, 0.99])      # P(L = 0 | G)

def draw(p, u): return int(np.argmax(u <= np.cumsum(p)))
def cond_D(g, i): w = np.array([PG[(i, d)][g] * PD[d] for d in (0, 1)]);                return w /
w.sum()
def cond_I(g, d): w = np.array([PG[(i, d)][g] * PS1[i] * PI[i] for i in (0, 1)]);       return w /
w.sum()
def cond_G(d, i): w = np.array([PL0[g] * PG[(i, d)][g] for g in range(3)]);             return w /
w.sum()

d, i, g = 0, 0, 1                       # evidence S = 1, L = 0 is held fixed
for u in [(0.80, 0.50, 0.70), (0.30, 0.20, 0.95)]:
    p = cond_D(g, i); d = draw(p, u[0]); print(p.round(3), d)
    p = cond_I(g, d); i = draw(p, u[1]); print(p.round(3), i)
    p = cond_G(d, i); g = draw(p, u[2]); print(p.round(3), g)
    print((d, i, g))
```

(a) The first sweep (lines 1–4): show the first two conditional distributions you computed by hand. [2]

(b) The final state after the second sweep. [1]

(c) Why are S and L never resampled, and why does `cond_D` not involve S or L? [1]

(d) After thousands of sweeps, what would the fraction of sweeps with I = 1 estimate? Name two practical issues. [1]

Question 28 · Week 11 · Rejection sampling (univariate) **(5 marks)**

We want samples from the density proportional to the unnormalised target $\tilde{p}(z) = 4z(1 - z)$ on $[0, 1]$, using the Uniform(0, 1) proposal $q(z) = 1$ and the rejection-sampling recipe from the Week 11 notebook: draw $z_0 \sim q$, draw $u_0 \sim \text{Uniform}(0, k \cdot q(z_0))$, accept z_0 if $u_0 < \tilde{p}(z_0)$. The random numbers are supplied explicitly. Predict the output.

```
import numpy as np

p_tilde = lambda z: 4 * z * (1 - z)         # unnormalised target on [0, 1]
q = lambda z: np.ones_like(z)               # proposal Uniform(0, 1): density 1
k = 1.0                                     # chosen so that k * q(z) >= p_tilde(z) for every z

z0 = np.array([0.10, 0.50, 0.85, 0.30, 0.95, 0.65, 0.20, 0.40])   # draws from the proposal q
u  = np.array([0.50, 0.90, 0.10, 0.80, 0.30, 0.45, 0.35, 0.99])   # independent Uniform(0, 1) draws
u0 = u * k * q(z0)                                                  # u0 ~ Uniform(0, k q(z0))
accept = u0 < p_tilde(z0)

print(p_tilde(z0).round(2))
print(accept.astype(int))
print(z0[accept], accept.mean())
print(round(z0[accept].mean(), 3))
print(round((2 / 3) / k, 3), round((2 / 3) / 2.0, 3))
```

(a) Give lines 1 and 2: the values $\tilde{p}(z_0)$ and the accept/reject indicators. Show how one accepted and one rejected draw are decided. [2]

(b) Give line 3 (accepted samples and the empirical acceptance rate). [1]

(c) Give line 4, and explain why the sample mean of the accepted values is a valid estimate of $E[Z]$ under the *normalised* target even though $\tilde{p}$ is unnormalised. [1]

(d) Line 5 prints the theoretical acceptance rates for $k = 1$ and $k = 2$. Derive them, and explain what happens to the samples if k is chosen too small (e.g. $k = 0.5$). [1]

Question 29 · Week 12 · Learning structure (PC) and parameters **(5 marks)**

Part 1 runs the skeleton phase of the PC algorithm with a p-value "oracle" ($\alpha = 0.05$; every test not in `table` has $p = 0.0005$ and therefore rejects independence). Part 2 estimates parameters from data.

```
import itertools

V = ['X', 'Y', 'Z', 'W']
alpha = 0.05
table = {(('X', 'Y'), ()): 0.70, (('W', 'X'), ('Z',)): 0.60, (('W', 'Y'), ('Z',)): 0.50}   # every
other test: p = 0.0005

def pvalue(a, b, S):
    return table.get((tuple(sorted((a, b))), tuple(sorted(S))), 0.0005)

edges = {frozenset(e) for e in itertools.combinations(V, 2)}
sepset = {}
for size in range(3):
    for e in sorted(edges, key=lambda e: sorted(e)):
        if e not in edges: continue
        a, b = sorted(e)
        cand = sorted(v for v in V if v not in e and (frozenset((a, v)) in edges or frozenset((b, v))
in edges))
        for S in itertools.combinations(cand, size):
            if pvalue(a, b, S) > alpha:
                edges.discard(e); sepset[(a, b)] = S
                break

print(sorted(map(sorted, edges)))
print(sepset)
colliders = []
for z in V:
    nb = sorted(v for v in V if frozenset((v, z)) in edges)
    for a, b in itertools.combinations(nb, 2):
        if frozenset((a, b)) not in edges and z not in sepset[(a, b)]:
            colliders.append((a, z, b))
print(colliders)
```

Part 2:

```
import pandas as pd
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.parameter_estimator import DiscreteMLE, DiscreteBayesianEstimator

df = pd.DataFrame({'A': [0, 0, 0, 0, 1, 1, 1, 1],
                   'B': [0, 0, 0, 1, 1, 1, 1, 1]})
m1 = DiscreteBayesianNetwork([('A', 'B')])
m1.fit(df, estimator=DiscreteMLE())
print(m1.get_cpds('B').values.round(3))

m2 = DiscreteBayesianNetwork([('A', 'B')])
m2.fit(df, estimator=DiscreteBayesianEstimator(prior_type='K2'))
print(m2.get_cpds('B').values.round(3))
```

(a) Part 1: give the three printed lines. [1]

(b) Orient the skeleton: identify the collider, then apply Meek's rule to the remaining edge, and give the final directed graph. [2]

(c) Part 2: give the two printed CPD tables and explain the danger of the first one when predicting an instance with A = 1, B = 0. [2]